In [1]:
```
import tensorflow as tf
import matplotlib.pyplot as plt
import pandas as pd

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense, Drop
```

In [2]:
```
VOCAB_SIZE = 10000
MAX_LENGTH = 200
EMBEDDING_DIM = 32
RNN_UNITS = 32

EPOCHS = 5
BATCH_SIZE = 128
```

In [3]:
```
(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=VOCAB_SIZE
)
```

In [4]:
```
x_train = pad_sequences(
    x_train,
    maxlen=MAX_LENGTH,
    padding="post",
    truncating="post"
)

x_test = pad_sequences(
    x_test,
    maxlen=MAX_LENGTH,
    padding="post",
    truncating="post"
)
```

In [6]:
```
def create_model(model_type):

    model = Sequential()

    model.add(
        Embedding(
            input_dim=VOCAB_SIZE,
            output_dim=EMBEDDING_DIM,
            input_length=MAX_LENGTH
        )
    )

    if model_type == "RNN":
        model.add(SimpleRNN(RNN_UNITS))

    elif model_type == "LSTM":
        model.add(LSTM(RNN_UNITS))

    elif model_type == "GRU":
        model.add(GRU(RNN_UNITS))

    model.add(Dropout(0.2))
    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

In [7]:
```
model_types = ["RNN", "LSTM", "GRU"]

models = {}
histories = {}
results = {}

for model_type in model_types:

    print("\n============================")
    print(model_type)
    print("============================")

    model = create_model(model_type)

    history = model.fit(
        x_train,
        y_train,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_split=0.2,
        verbose=1
    )

    test_loss, test_accuracy = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    models[model_type] = model
    histories[model_type] = history

    results[model_type] = [
        test_loss,
        test_accuracy
    ]

    print("Test Loss:", test_loss)
    print("Test Accuracy:", test_accuracy)
```

```
============================
RNN
============================
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
```

```
C:\Users\cs2\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core
\embedding.py:123: UserWarning: Argument `input_length` is deprecated. Just remov
e it.
  warnings.warn(
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.5094 - loss: 0.6950 - val
_accuracy: 0.5238 - val_loss: 0.6910
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.6148 - loss: 0.6538 - val
_accuracy: 0.5090 - val_loss: 0.6974
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - accuracy: 0.6852 - loss: 0.5702 - val
_accuracy: 0.5092 - val_loss: 0.7460
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - accuracy: 0.7621 - loss: 0.4462 - val
_accuracy: 0.5172 - val_loss: 0.8109
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - accuracy: 0.8092 - loss: 0.3719 - val
_accuracy: 0.5066 - val_loss: 0.8705
Test Loss: 0.8833997845649719
Test Accuracy: 0.5018399953842163

==============================
LSTM
==============================
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 9s 50ms/step - accuracy: 0.5143 - loss: 0.6925 - val
_accuracy: 0.5440 - val_loss: 0.6880
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 8s 48ms/step - accuracy: 0.6600 - loss: 0.6392 - val
_accuracy: 0.5160 - val_loss: 0.7389
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 8s 48ms/step - accuracy: 0.5670 - loss: 0.6780 - val
_accuracy: 0.7010 - val_loss: 0.6127
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 8s 48ms/step - accuracy: 0.5588 - loss: 0.6836 - val
_accuracy: 0.5668 - val_loss: 0.6758
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 8s 48ms/step - accuracy: 0.6445 - loss: 0.6562 - val
_accuracy: 0.5658 - val_loss: 0.6749
Test Loss: 0.6804606914520264
Test Accuracy: 0.5618399977684021

==============================
GRU
==============================
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 10s 57ms/step - accuracy: 0.5079 - loss: 0.6927 - va
l_accuracy: 0.5312 - val_loss: 0.6920
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 9s 55ms/step - accuracy: 0.5623 - loss: 0.6885 - val
_accuracy: 0.5176 - val_loss: 0.6885
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 9s 56ms/step - accuracy: 0.5993 - loss: 0.6557 - val
_accuracy: 0.6014 - val_loss: 0.6060
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 9s 55ms/step - accuracy: 0.5788 - loss: 0.7000 - val
_accuracy: 0.5410 - val_loss: 0.6785
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 9s 55ms/step - accuracy: 0.6250 - loss: 0.6207 - val
_accuracy: 0.6472 - val_loss: 0.6512
Test Loss: 0.6654340028762817
Test Accuracy: 0.6311600208282471
```

In [13]:
```
results_df = pd.DataFrame(
    results,
    index=["Test Loss", "Test Accuracy"]
).T

print("\nPerformance Comparison")
print(results_df)

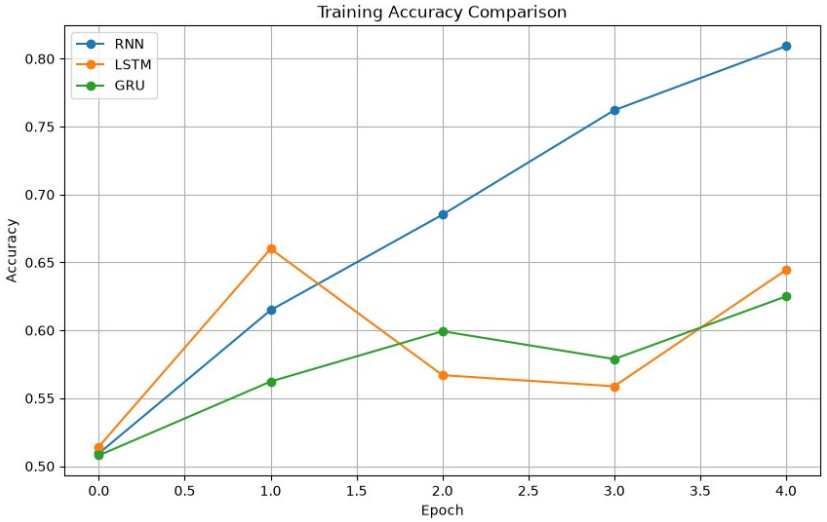

plt.figure(figsize=(10, 6))

for model_type in model_types:

    plt.plot(
        histories[model_type].history["accuracy"],
        marker="o",
        label=model_type
    )

plt.title("Training Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

```
Performance Comparison
      Test Loss  Test Accuracy
RNN    0.883400        0.50184
LSTM   0.680461        0.56184
GRU    0.665434        0.63116
```

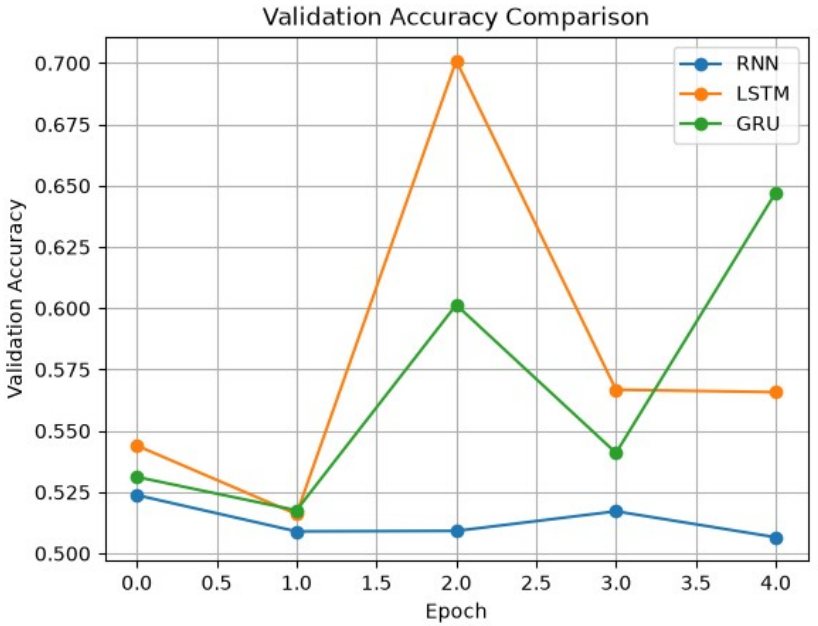


In [10]:
```
for model_type in model_types:

    plt.plot(
        histories[model_type].history["val_accuracy"],
        marker="o",
        label=model_type
    )

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()


plt.figure(figsize=(10, 6))
```



Out[10]:
```
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
```

In [12]:
```
plt.figure(figsize=(10, 6))

for model_type in model_types:

    plt.plot(
        histories[model_type].history["val_loss"],
        marker="o",
        label=model_type
    )

plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```

```python
accuracy = [
    results[m][1]
    for m in model_types
]

plt.figure(figsize=(8, 5))

bars = plt.bar(
    model_types,
    accuracy,
    color=["steelblue", "orange", "green"]
)

plt.title("Test Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Test Accuracy")
plt.ylim(0, 1)

for bar, value in zip(bars, accuracy):

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

plt.show()


loss = [
    results[m][0]
    for m in model_types
]

plt.figure(figsize=(8, 5))

bars = plt.bar(
    model_types,
    loss,
    color=["steelblue", "orange", "green"]
)

plt.title("Test Loss Comparison")
plt.xlabel("Model")
plt.ylabel("Test Loss")

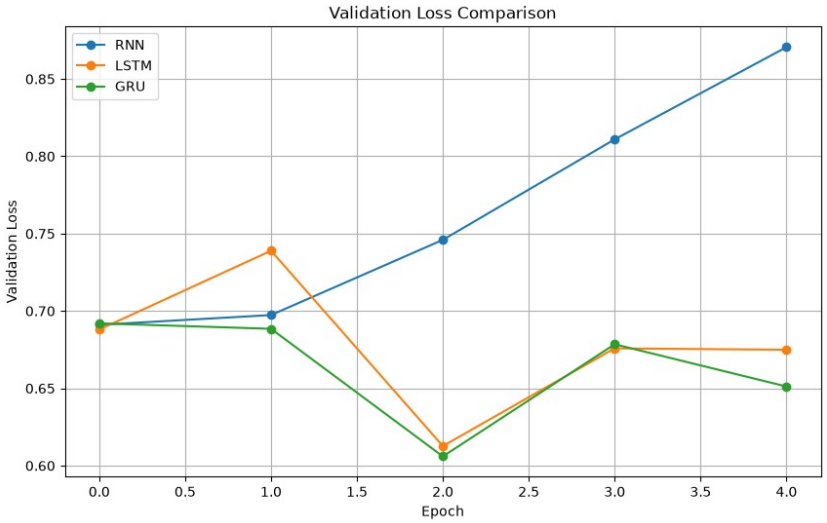
for bar, value in zip(bars, loss):

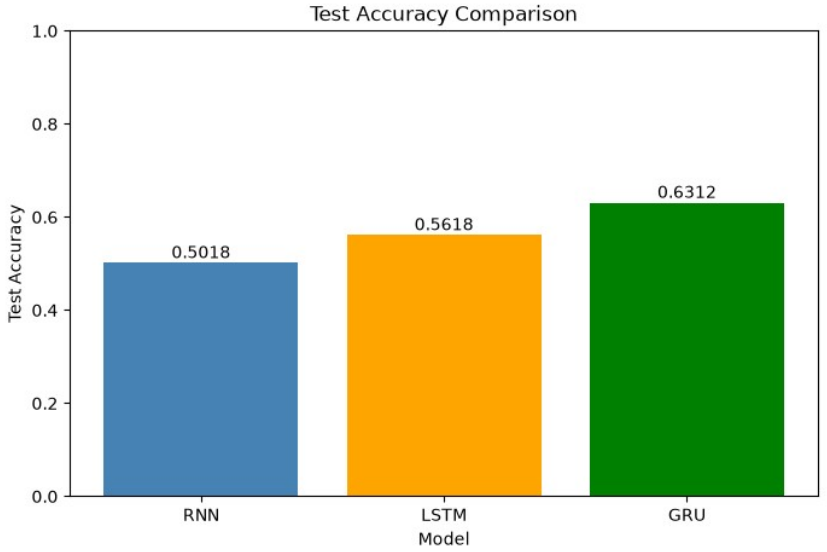
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

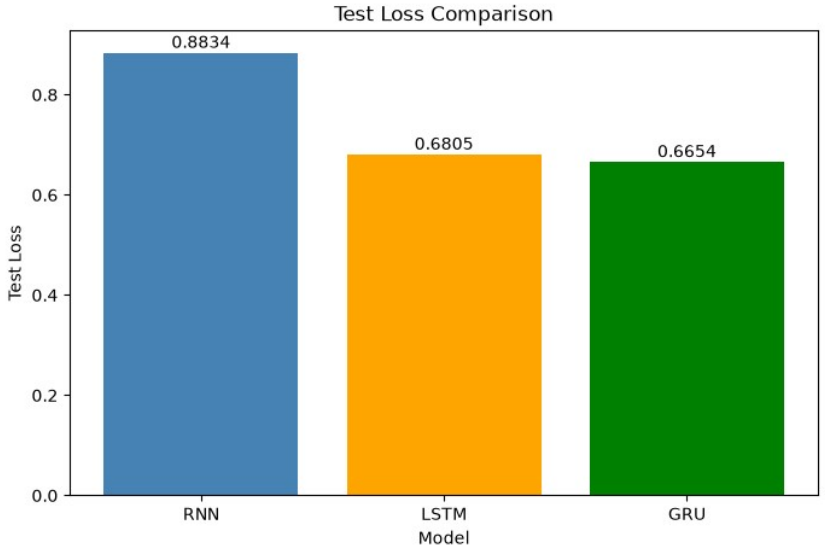
plt.show()
```

```python
for model_type in model_types:

    print(
        model_type,
        "Parameters:",
        models[model_type].count_params()
    )
```








```
RNN Parameters: 322113
LSTM Parameters: 328353
GRU Parameters: 326369
```

In [ ]: